Predictive Deadlock Warning: Early Detection of Unsafe States from Resource-Request Patterns

Revanth Reddy Lingala
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India

Padigela Srujan
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India

***Abstract*—Banker's algorithm decides whether a resource-allocation state is safe now; it gives no signal that a running system is drifting towards an unsafe state, and no study has measured how early, how reliably and at what cost that transition can be anticipated. We measured it in a discrete-event simulator that grants every valid request, so that unsafe states can occur, over three synthetic workloads and systems of 10 to 200 processes. We compared a rule-based score over five resource-pressure indicators with a free-resource threshold, single indicators, logistic regression and gradient-boosted trees on the same signals, a scalar safety margin, a partial and a full one-step lookahead, using event-level alarm precision, event recall and lead time on 150,000 test sequences (338.3 million simulation steps) disjoint from 45,000 tuning sequences. In these configurations the rule-based score reached F1 0.28 with a median warning of 10 steps, against 0.16 for the free-resource threshold and 0.006 for random alarms at a matched rate, and was ahead of both in all 20 of 20 independent replications; gradient-boosted trees on the same signals reached 0.48. On the sequences where all methods were computed, the safety margin reached F1 0.72 and full lookahead 0.74, at 1.7 and 43.5 times the compute of one simulation step for 200 processes in our NumPy implementation; the margin was lower by 0.027 (95% interval 0.023–0.031) and equivalent within a post-hoc margin of ± 0.05, but both warned later (median 4 steps). The results describe one simulator, synthetic workloads with declared maximum claims and one implementation; real allocation traces and biased misdeclaration of claims remain untested.**

***Index Terms*—deadlock avoidance, Banker's algorithm, early warning, resource allocation, event-based evaluation, operating systems**

I. Introduction

Deadlock remains a practical hazard wherever resources are allocated incrementally, as in large cluster managers [1]. Cluster schedulers that place the tasks of a gang-structured job one at a time can leave several jobs each holding part of what they need, a "temporary deadlock" reported in deep-learning platforms [2], [3] and the motivation for all-or-nothing placement in recent Kubernetes releases [4]; resource exhaustion more broadly is a recurring cause of cloud outages [5]. Banker's algorithm [6], [7], [8], the classical avoidance method, answers a present-tense question: is the state safe now? It is seldom deployed because of its cost [9], and it gives operators no advance notice of the kind monitoring practice expects [10].

What is known falls into three groups. The theory and complexity of avoidance are well developed [7], [8], [11], [12]. Safety tests have been made faster or incremental [9], [13], [14]. "Deadlock prediction" has been studied in other senses: lock-order deadlocks in program traces [15], [16] and overload of virtual machines in a cloud simulator [17]. What is not known is the lead time, reliability and compute cost of anticipating the outcome of Banker's safety algorithm in a running allocator, before it reports an unsafe state.

A simple estimate suggests the problem is easy. Unsafety is a deterministic function of the allocation state, so any aggregate of outstanding need against free units should rise before the boundary is crossed, and a per-step safety check costs $\Theta(mn^2)$, affordable for small systems. The estimate does not say how far ahead such an aggregate rises, at what false-alarm rate, which aggregate works, how warning quality depends on the workload, or how the cost of Banker-based methods grows with n. These are the quantities we measured.

Our thesis is that, in these configurations, lightweight indicators anticipate unsafe states about ten steps ahead with moderate precision; that non-linear combinations of the same signals do substantially better; and that a safety check with a fixed number of safety-algorithm runs per step comes within 0.03 F1 of full lookahead at a small fraction of its cost for large systems, but warns later than the lightweight indicators. Observations that we cannot explain are reported in Section V-H.

The hypotheses below were formulated after two earlier, exploratory runs (Table IV) and were not pre-registered; Section IV-C lists the resulting deviations.

- **H1** (advance warning): a rule-based indicator score raises alarms before unsafe transitions with F1 above a matched-rate random baseline and a free-resource threshold (Section V-A).
- **H2** (equivalence at lower cost): the safety margin, which uses a fixed seven safety checks per step, is equivalent in F1 to full lookahead within $\pm\delta$, $\delta = 0.05$ (Section V-C).
- **H3** (claim robustness): F1 of claim-based indicators is insensitive to unbiased per-entry claim error (Section V-E). The companion hypothesis, that biased under-claiming degrades them, was not tested.

Contributions.

C1 Measured alarm precision, event recall and lead time for twelve warning methods on 150,000 test sequences, with the workload regimes in which warning quality holds or fails, and paired comparisons over independent

replications (Sections V-A–V-B).

C2 The cost–accuracy–lead-time frontier for anticipating the outcome of Banker's safety algorithm, with compute measured per method and identified as a property of our implementation, and an equivalence test between the safety margin and full lookahead (Section V-C).

C3 An evaluation protocol for deadlock early warning adapted from clinical and seismic early warning, implemented in a discrete-event simulator verified by 134 tests (Sections III–IV).

Secondary results are the unsafe-versus-deadlock outcomes as a property of the model (Section V-D), horizon sensitivity and the scaling of the vectorised safety check. The simulator and pipeline are an artefact, not a contribution in themselves.

Scope. The study uses one simulator, three synthetic workloads with declared maximum claims and a fixed set of processes per sequence, one NumPy implementation and one AWS instance type. It does not use real allocation traces, does not act on warnings (no avoidance policy is evaluated) and does not consider distributed allocators.

II. Background and Related Work

A. Banker's Algorithm, Unsafe States and Deadlock

A system has m resource types with $total_j$ units each and n processes. Process i declares a maximum claim Max_{ij}, holds $Alloc_{ij}$ and may still request $Need_{ij} = Max_{ij} - Alloc_{ij}$; free units are $Avail_j = total_j - \sum_i Alloc_{ij}$. A state is *safe* if a *safe sequence* of all processes exists, that is, an order in which the remaining need of each process can be met from the available resources plus the resources released by the processes earlier in the sequence [7], [8]; the textbook check costs $\Theta(mn^2)$ [18], [19], [9]. Deadlock can occur only if the four Coffman conditions hold simultaneously: mutual exclusion, hold and wait, no preemption and circular wait [20]; it is detected by reducing the resource-allocation graph [21]. The safety check is pessimistic (every process may request its full claim); detection is optimistic (running processes finish). A state can therefore be unsafe without being deadlocked, but never deadlocked while safe; safety also does not guarantee liveness [22], [23]. Dijkstra's definition of an unsafe state is the ground truth for every metric in this paper.

B. Related Measurements

Table I positions this study; its entries describe the scope of each work, not shortcomings. The theory of avoidance treats "deadlock prediction" as a static decision about whether a system of processes can deadlock and establishes its complexity [11], [12], [24]; it does not observe a running allocator. Faster or dynamic safety and detection algorithms [9], [13], [25], [26], [14] reduce the cost of answering the present-tense question; two related works supply our Banker-based baselines, the minimum resources needed to stay safe [13] and the safety of prospective requests [27]. Prevention by resource ordering [28] removes circular wait altogether, at the price of constraining request order. Detection in databases and distributed systems [29], [30], [31], [32] acts after a deadlock forms. Program analysis predicts lock-order deadlocks from execution traces [33], [34], [16], [15], a different object from resource-allocation safety. Learning-based work forecasts overload in a cloud simulator [17] or proposes AI-based thread-deadlock prediction without an evaluation [35]. Petri-net deadlock control in manufacturing [36], [37], [38], [39] enforces liveness by construction. None of the works we found reports the lead time of a warning before Banker's algorithm declares a state unsafe.

The related work was assembled in October 2026 from keyword searches combining *Banker's algorithm*, *deadlock prediction*, *deadlock avoidance*, *unsafe state* and *early warning*, made with an AI-assisted research tool, and from the reference lists of the papers found; every entry was checked against its Crossref record. This was not a systematic review. We did not search IEEE Xplore, the ACM Digital Library or Google Scholar exhaustively, so the absence of a direct predecessor in Table I is a statement about what we found, not a claim that none exists.

C. Early-Warning Evaluation Method

Clinical early warning faces the same structure as our problem—a rare harmful state change under continuous monitoring, valuable only if anticipated—and evaluates it with event-level recall, alarm precision and alarm silencing [40]. Earthquake early warning makes the trade-off between warning time and precision explicit and counts an alarm raised at the event itself as a miss [41]. Range-based and window-based scoring [42], [43], precision–recall analysis for rare events [44], [45], bootstrap intervals [46] and the anomaly-detection and failure-prediction literature [47], [48] complete the toolkit we adopt. For equivalence we use two one-sided tests on paired differences.

III. System Model and Experimental Design

If Banker's algorithm refused unsafe requests, the system would never become unsafe and nothing could be predicted. The simulator therefore uses *permissive allocation with passive monitoring*: every valid request is granted, and Banker's safety algorithm and the deadlock-detection algorithm are used only to monitor the state. Each step (1) retries waiting processes first-come first-served, finishing any whose claim is now met and repeating until nothing changes; (2) lets one active, non-waiting process issue a request no larger than its need, granted if it fits and otherwise blocking the process; (3) releases the resources of completed processes; (4) in the normal workload, allows an early partial release; and (5) records whether the state is unsafe or deadlocked, the indicators and the Banker-based baselines. A sequence ends at the first deadlock, when all processes finish, or at a step cap. Because no request is refused, passive monitoring also avoids the clinical confound in which interventions turn correct alarms into apparent false positives [40].

The primary prediction target is the *unsafe transition*, a step at which the state changes from safe to unsafe; transitions after fewer than five safe steps are merged into the previous

TABLE I
SCOPE OF RELATED WORK. ✓ = COVERED; – = OUTSIDE THE STUDY'S SCOPE OR NOT REPORTED. P = PEER-REVIEWED, A = PREPRINT. ENTRIES DESCRIBE SCOPE, NOT SHORTCOMINGS.

Study	Type	Setting	Banker safety	Forecast of transition	Lead time	Event-based eval.	Cost per method
Dijkstra [7]; Habermann [8]	P	Avoidance by safety test	✓	–	–	–	–
Gold [11]; Minoura [12]; Kameda [24]	P	Complexity of deciding avoidability	✓	–	–	–	complexity
Belik [9]; Finkel [13]; Zohora [14]	P	Faster or dynamic safety tests	✓	–	–	–	✓
Andrews & Levin [27]	P	Safety of prospective requests	✓	–	–	–	–
Agrawal [29]; Chandy [30]	P	Deadlock detection (databases, distributed)	–	–	–	–	✓
Kalhauge [16]; Tunç [15]	P	Lock-order deadlocks in program traces	–	–	–	–	✓
Le Ngoc [17]	P	ML load balancing in CloudSim	–	✓	–	–	–
Oluwaseyi [35]	A	AI thread-deadlock prediction (proposal)	–	✓	–	–	–
Ezpeleta [37]; Li [39]	P	Petri-net deadlock control (manufacturing)	–	–	–	–	–
Hyland [40]; Münchmeyer [41]	P	Clinical / seismic early warning	–	✓	✓	✓	–
This work		Early warning of Banker-unsafe states	✓	✓	✓	✓	✓

unsafe episode [42], and transitions in the first three steps are excluded for all methods. The secondary target is the *deadlock occurrence*.

Requests are fractions of remaining need, so behaviour is comparable across sizes; the overcommitment ratio $\sum_i Max_{ij}/total_j$ is the main calibration parameter (1.43, 1.58 and 1.48 realised; Table V). *Normal* issues small single-resource requests with occasional partial releases. *Bursty* switches, once a random 20–60% of initial need has been requested, to 80% of processes grabbing 30–60% of their remaining need, capped by free units, akin to piecewise gang placement [2]. *Deadlock-prone* makes process i prefer resource $(i{+}1) \bmod m$, inducing circular wait [20]. Parameters were calibrated on tuning seeds so that each workload mixes sequences with and without unsafe events (35%, 60% and 62% of test sequences contain at least one).

Out of scope are real allocation traces, process arrivals beyond the initial set, avoidance that acts on a warning, distributed or multi-node allocators, preemption and recovery, processes that stop short of their claims, and adversarial request streams.

Table II lists the methods. The five indicators, each in $[0, 1]$, are *resource utilisation* (the fraction of all units that are allocated), *demand ratio* $\frac{1}{m}\sum_j S_j/(S_j + Avail_j)$ with S_j the outstanding need of active processes, *resource contention* (the fraction of resource types for which at least two active processes need more than is free), *blocked-process ratio* (the fraction of active processes that are blocked) and *request rate* (the clipped relative increase in units requested over the last ten steps). The rule-based score is their weighted mean on a 0–100 scale. Every method uses the same alarm rule: a transition at step t if the score has been at or above a threshold for k consecutive steps and no transition occurred in the previous R steps [40]; alarms are suppressed while the state is already unsafe. The learned models, logistic regression and gradient-boosted trees [49] ("boosted trees" in the tables), use the five indicators and their ten-step changes. The safety margin is a scalar, proportional form of Finkel and Madduri's measure [13], not their algorithm. Table III separates what this paper claims from what it does not.

TABLE II
METHODS COMPARED. COST IS EXTRA COMPUTE PER STEP RELATIVE TO ONE SIMULATION STEP AT 200 PROCESSES (TABLE XI).

Method	Signal	Trained	Cost
Random	Bernoulli, matched rate	no	≈0
Free-resource threshold	fraction of resources available	no	≈0
Single indicators (4)	demand ratio, contention, blocked-process ratio or request rate	no	≈0
Rule-based (ours)	weighted mean of 5 indicators	no	≈0
Rule-based, max	weighted maximum	no	≈0
Logistic regression	5 indicators + 10-step changes	yes	≈0
Boosted trees	same features	yes	≈0
Safety margin	binary search over free units (7 checks)	no	1.66
Partial lookahead	≤3 prospective-grant checks	no	0.91
Full lookahead	1 check per running process	no	43.45

IV. Implementation and Methodology

A. Implementation

The simulator is written in Python with a NumPy-vectorised safety check [50] that finishes every process that fits in each pass; it agrees with the textbook loop on 2,000 random states, as does deadlock detection, and both reproduce the textbook examples [18]. Detection is event-driven: it reruns only when

TABLE III
SCOPE OF CLAIMS

Class	Claimed	Not claimed
Correctness	Simulator invariants; vectorised = reference safety check and detection on 2,000 random states; deadlocked $\Rightarrow$ unsafe (134 tests)	A formal proof of the simulator
Warning accuracy	Precision, recall, F1 and lead time in these workloads and sizes	Accuracy on real systems or other workload families
Replication	Rankings hold in 20 of 20 independent replications	Stability under re-tuning or a different generator
Equivalence	Safety margin within ± 0.05 F1 of full lookahead (δ post hoc)	Equivalence at tighter margins; "no difference"
Cost	Per-step compute of our NumPy implementation	Cost of other implementations or incremental tests
Deadlock outcome	Fraction of unsafe episodes ending in deadlock in this simulator	That unsafe states usually deadlock in real systems
Robustness	Insensitivity to unbiased $\pm 20\%$ per-entry claim error	Robustness to biased misdeclaration

the waiting set or a pending request changes, which we verified against every-step detection. Invariants (conservation of units, allocation $\leq$ claim, pending $\leq$ need, finished processes hold nothing) are asserted in a debug mode. The Banker-based baselines draw on a separate random stream, so the simulated workload is identical with or without them. The 134 automated tests cover every row of our edge-case matrix. The main run used Python 3.12.3, NumPy 2.4.6, pandas 3.0.6, PyArrow 25.0.1 and scikit-learn 1.9.1. The analysis scripts generate every number and table in this paper.

B. Testbed and Workloads

We ran the pipeline three times (Table IV): a pilot on a laptop to check that it worked end to end, an exploratory cloud run with a smaller set of methods, and the main run. Every result in this paper comes from the main run; the earlier runs only shaped the hypotheses and are never pooled with it. Each workload ran at five sizes (10–200 processes, 5–20 resource types) with 10,000 test seeds and 3,000 tuning seeds per cell (Table V); the tuning seed range (0–99,999) and test seed range (from 100,000) are disjoint, which the code asserts. The main run ran headless on one AWS EC2 c7i.8xlarge instance (32 vCPUs, 62 GB, region ap-south-2, Ubuntu 24.04) with 32 worker processes and one BLAS/OpenMP thread each; the whole pipeline took 5 h 40 min and cost US$8.39.

C. Statistical Analysis

An alarm at step a is true if a transition falls in $[a{+}1, a{+}H_{\max}]$, $H_{\max} = 20$; a transition is captured by the earliest unused alarm in its window, which gives its lead time [40], and an alarm at the transition step is a miss [41]. Alarms in the last $H_{\max}$ steps of a sequence cut by the step cap

TABLE IV
EXPERIMENTAL RUNS. ONLY RUN 3 IS REPORTED; RESULTS ARE NEVER POOLED ACROSS RUNS.

Run	Machine	Sequences	Steps	Purpose
1. Pilot	laptop	48,000	55.7 M	pipeline check; not reported
2. Exploratory	AWS EC2, 32 vCPUs	300,000	676.2 M	fewer methods; not reported
3. Main	AWS EC2, 32 vCPUs	195,000	439.6 M	all 12 methods; all results

are censored. We report micro-averaged alarm precision, event recall, F1, false alarms per 1,000 steps, lead-time distributions and an NAB-style score [43]. For each method the threshold (5–95), $k \in \{1, 2, 3\}$, $R \in \{10, 20\}$ and, for combinations, the weight vector were chosen on tuning seeds to maximise F1 subject to at most five false alarms per 1,000 steps and a median lead of at least three steps; the learned models were trained on even and thresholded on odd tuning seeds, with a prior correction to the true positive rate.

Experimental unit. A percentile bootstrap over sequences, stratified by workload and size [46], gives intervals such as F1 0.28 [0.28, 0.28]. These capture only the variation from sampling sequences out of one generator; they are not intervals over replications. We therefore also deal the test seeds of each (workload, size) cell round-robin into 20 batches, so that each batch is a stratified replicate of the test split, pool F1 within each batch and compare methods by paired per-batch differences with t intervals over the 20 batches. With 10 or 50 batches the mean differences are unchanged and the interval bounds move by at most 0.001. Thresholds are the ones tuned once, so batches replicate test-side sampling only, not tuning or the choice of generator.

Equivalence. H2 is tested with two one-sided tests on the paired batch-level F1 difference against $\pm\delta$, $\delta = 0.05$: equivalent if the 90% interval of the mean difference lies inside $(-\delta, \delta)$, otherwise inconclusive. We also report the smallest δ at which equivalence would hold.

Deviations and post-hoc choices. (1) The tuning split was reduced from 10,000 to 3,000 seeds per cell after a pilot projected the run past its time limit. (2) Tuning, evaluation and figures were re-run after two bugs found mid-run (thread oversubscription in the learned-model workers; a NaN in the prior correction of learned scores); the simulation data were unaffected. (3) The bursty workload was redesigned after calibration showed the first design produced no unsafe states. (4) The paired lead-time comparison, the NAB score, the per-size table, the batch analysis, the common-subset deadlock table and the equivalence test were chosen after the main run's headline results were seen. (5) H1–H3 and $\delta = 0.05$ were written after the point estimates of all three runs were known. No analysis was pre-registered; all results should be read as exploratory.

TABLE V
TEST SPLIT PER WORKLOAD AND SYSTEM SIZE (n PROCESSES, m RESOURCE TYPES). TEST SPLIT: 150,000 SEQUENCES, 338.3 M STEPS; TUNING SPLIT (NOT SHOWN): 45,000 SEQUENCES, 101.3 M STEPS.

Workload	n	m	Sequences	Steps	Unsafe transitions	% sequences with a transition	% ending in deadlock	Overcommitment ratio*
Normal	10	5	10,000	1,071,547	2,754	26.9	25.4	1.43
Normal	25	8	10,000	4,494,155	1,674	15.7	14.3	1.43
Normal	50	10	10,000	11,527,074	1,270	11.3	10.1	1.43
Normal	100	20	10,000	39,280,274	8,469	65.8	62.7	1.43
Normal	200	20	10,000	82,123,372	7,358	56.2	53.6	1.43
Bursty	10	5	10,000	707,662	7,066	70.7	70.7	1.58
Bursty	25	8	10,000	2,939,715	5,954	59.5	59.5	1.58
Bursty	50	10	10,000	7,434,425	5,317	53.2	53.2	1.58
Bursty	100	20	10,000	27,023,589	6,185	61.9	61.9	1.58
Bursty	200	20	10,000	56,973,051	5,346	53.5	53.5	1.58
Deadlock-prone	10	5	10,000	1,020,230	4,073	40.7	40.7	1.48
Deadlock-prone	25	8	10,000	4,036,769	4,047	40.5	40.5	1.48
Deadlock-prone	50	10	10,000	9,877,410	4,513	45.1	45.1	1.48
Deadlock-prone	100	20	10,000	29,793,804	9,316	93.2	93.2	1.48
Deadlock-prone	200	20	10,000	59,959,348	9,270	92.7	92.7	1.48

* Total declared maximum claims divided by total units; above 1 means processes together may claim more than exists.

V. Results

A. Per-Method Accuracy and Lead Time

Table VI reports every method except full lookahead on the full test population. Full lookahead was computed only on a subset of sequences, so it is compared with all other methods on that common subset in Table VIII.

The rule-based score reached F1 0.280 with a median lead of ten steps, against 0.162 for the free-resource threshold and 0.006 for matched-rate random alarms. With the replication as the unit (Table VII), it was ahead of the free-resource threshold by 0.118 [0.115, 0.121] and of random by 0.274 [0.270, 0.278] in all 20 replications, which supports H1. The batch-level standard deviation of the rule-based F1 (0.0075 for batches of 7,500 sequences) implies a standard error of the full-population estimate of about 0.002, consistent with the width of the bootstrap interval; resampling sequences did not hide variation between replicates of this kind.

Our initial expectation, that combining the five indicators would beat every single indicator, was wrong. Contention alone reached 0.338 and was ahead of the combination in all 20 replications (by 0.058 [0.055, 0.062]). The ablation (Table IX) shows why: removing contention lowered the equal-weight mean from 0.24 to 0.20, while removing blocked-process ratio raised it to 0.27; averaging dilutes the strongest indicator. The tuned weights double contention, under which the weighted maximum coincides with contention alone, so its identical row is a design artefact. Boosted trees on the same signals reached 0.477, ahead of the combination by 0.197 [0.193, 0.201]: the lightweight indicators carry more information than a weighted mean extracts.

Finding 1. In these configurations the rule-based score warned a median 10 steps ahead with F1 0.28, ahead of the free-resource threshold (0.16) and matched-rate random alarms (0.006) in 20 of 20 independent replications. Contention alone (0.34) and boosted trees on the same signals (0.48) did better.

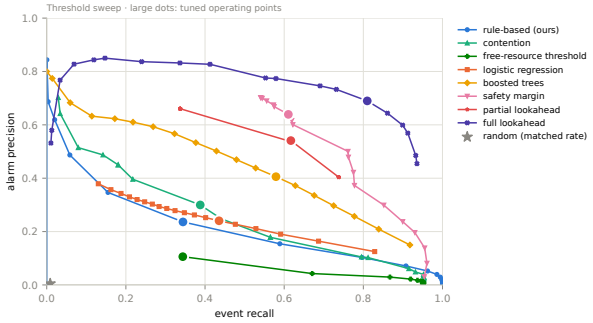


Fig. 1. Precision–recall along each threshold sweep (test split).

B. Where Warning Quality Varies

Warning quality depended on how danger built up (Table X). The rule-based score reached F1 0.42 on bursty workloads but 0.22 and 0.23 on the gradual normal and deadlock-prone workloads. In bursty sequences, unsafe transitions followed a median 26 steps after the burst began and 3.0% occurred before it, so the abrupt change in request pattern gives the lightweight indicators time to react. The Banker-based methods kept a large advantage on the gradual workloads (safety margin 0.55 and 0.54). F1 of most methods was lower at $n = 200$ than at small sizes (observation A4, Section V-H), and blocked-process ratio reversed its usual standing on bursty workloads (observation A1).

Finding 2. Warning quality depended on the workload: the rule-based score reached F1 0.42 on bursty workloads and 0.22–0.23 on gradual ones, where Banker-based methods kept a large advantage.

C. Cost–Accuracy Frontier

Table XI and Fig. 2 relate accuracy to compute. One safety check took 59.9 μs on average and one simulation step 289.2 μs. The safety margin's seven checks cost 1.66 simulation steps at $n = 200$ and the partial lookahead 0.91;

TABLE VI
PER-METHOD RESULTS ON THE FULL TEST POPULATION (PREDICTING UNSAFE TRANSITIONS UP TO 20 STEPS AHEAD; 150,000 SEQUENCES, 82,612 TRANSITIONS). FALSE ALARMS PER 1,000 STEPS; LEAD IN STEPS; NAB SCORE: 0 = SILENT, 100 = PERFECT EARLIEST DETECTION. INTERVALS: 95% BOOTSTRAP OVER SEQUENCES. FULL LOOKAHEAD IS IN TABLE VIII.

Method	Precision	Recall	F1 [95% CI]	False alarms	Median lead	NAB score
Rule-based (ours)	0.236	0.344	0.280 [0.277, 0.283]	0.333	10	22.1
Contention	0.300	0.388	0.338 [0.336, 0.341]	0.221	7	27.2
Rule-based, max	0.300	0.388	0.338 [0.336, 0.341]	0.221	7	27.2
Demand ratio	0.314	0.136	0.190 [0.187, 0.193]	0.072	7	9.5
Blocked-process ratio	0.242	0.259	0.250 [0.248, 0.252]	0.307	13	16.6
Request rate	0.161	0.116	0.135 [0.133, 0.137]	0.148	9	6.6
Free-resource threshold	0.106	0.344	0.162 [0.160, 0.164]	0.708	8	13.0
Logistic regression	0.240	0.435	0.310 [0.307, 0.312]	0.480	12	28.3
Boosted trees	0.406	0.579	0.477 [0.475, 0.480]	0.292	11	45.0
Safety margin	0.639	0.611	0.625 [0.622, 0.627]	0.084	4	43.8
Partial lookahead	0.541	0.617	0.576 [0.574, 0.579]	0.128	5	45.6
Random (matched rate)	0.005	0.009	0.006 [0.006, 0.006]	0.438	11	–

TABLE VII
F1 DIFFERENCES BETWEEN TWO METHODS (FIRST MINUS SECOND), COMPUTED SEPARATELY IN EACH OF 20 INDEPENDENT REPLICATIONS (95% INTERVAL OVER REPLICATIONS). WINS = REPLICATIONS IN WHICH THE FIRST METHOD IS AHEAD. SET: "ALL" = FULL TEST SET, "SUBSET" = TABLE VIII.

Comparison	Set	F1 difference [95% CI]	Wins
Rule-based vs random	all	$+0.274$ [0.270, 0.278]	20/20
Rule-based vs free-resource threshold	all	$+0.118$ [0.115, 0.121]	20/20
Rule-based vs contention	all	-0.058 [-0.062, -0.055]	0/20
Rule-based vs logistic regression	all	-0.030 [-0.033, -0.027]	0/20
Rule-based vs boosted trees	all	-0.197 [-0.201, -0.193]	0/20
Rule-based vs safety margin	all	-0.344 [-0.348, -0.341]	0/20
Safety margin vs full lookahead	subset	-0.027 [-0.031, -0.023]	0/20
Partial lookahead vs full lookahead	subset	-0.052 [-0.055, -0.049]	0/20
Boosted trees vs full lookahead	subset	-0.206 [-0.212, -0.200]	0/20
Rule-based vs full lookahead	subset	-0.302 [-0.311, -0.294]	0/20

TABLE VIII
ALL METHODS ON THE COMMON SUBSET WHERE FULL LOOKAHEAD WAS COMPUTED (61,350 SEQUENCES: ALL WITH $n \leq 25$, 200 SEEDS PER CELL FOR $n = 50, 100$, 50 FOR $n = 200$; 26,382 TRANSITIONS). FALSE ALARMS PER 1,000 STEPS; LEAD IN STEPS.

Method	F1 [95% CI]	False alarms	Median lead
Full lookahead	0.745 [0.741, 0.749]	0.624	4
Safety margin	0.718 [0.714, 0.722]	0.543	4
Partial lookahead	0.693 [0.689, 0.698]	0.683	5
Boosted trees	0.539 [0.535, 0.543]	2.394	10
Contention	0.515 [0.510, 0.520]	1.039	6
Rule-based (ours)	0.443 [0.437, 0.448]	0.481	5
Logistic regression	0.362 [0.359, 0.365]	5.442	12
Request rate	0.333 [0.327, 0.338]	0.516	8
Free-resource threshold	0.298 [0.292, 0.303]	0.351	4
Demand ratio	0.182 [0.176, 0.187]	0.084	4
Blocked-process ratio	0.141 [0.136, 0.146]	0.320	4
Random	0.014 [0.013, 0.014]	0.422	9

TABLE IX
ABLATION: RULE-BASED SCORE WITH EQUAL WEIGHTS AND ONE INDICATOR REMOVED (EACH RE-TUNED). FALSE ALARMS PER 1,000 STEPS.

Indicators used	Precision	Recall	F1	False alarms
All five	0.17	0.40	0.24	0.59
Without resource utilisation	0.19	0.39	0.26	0.47
Without demand ratio	0.17	0.45	0.25	0.64
Without contention	0.14	0.36	0.20	0.65
Without blocked-process ratio	0.23	0.31	0.27	0.30
Without request rate	0.15	0.39	0.22	0.54

their number of checks does not grow with n. Full lookahead, one check per running process, cost 43.45 at $n = 200$. These costs are specific to our implementation—a NumPy-vectorised check in Python—not properties of the algorithms: an incremental safety test [9], [13] or a compiled implementation would move every Banker-based point, so the ratios, not the absolute times, are the result, and even the ratios are untested on a second implementation.

On the common subset, the safety margin was lower than full lookahead by 0.027 [0.023, 0.031] in all 20 replications; the difference is not zero. Its 90% interval, $[-0.030, -0.024]$, lies inside ± 0.05, so the two are equivalent at that margin ($p_{\text{TOST}} < 0.001$); they would be equivalent at any δ above 0.030, and not below. The partial lookahead was lower by 0.052 and is not equivalent at $\delta = 0.05$ ($p_{\text{TOST}} = 0.87$). Since δ was fixed after the point estimates were known, H2 is supported only as an exploratory result.

The Banker-based methods bought their accuracy with later warnings (Fig. 3). Their median lead was 4–5 steps against 7–13 for the lightweight indicators. On the 17,648 transitions that both captured, the rule-based score warned a median 9 steps ahead and the safety margin 3; on the 8,012 transitions captured by both it and full lookahead, 5 and 4 steps.

TABLE X
F1 PER WORKLOAD (UNSAFE TRANSITIONS, 20-STEP HORIZON).

Method	Normal	Bursty	Deadlock-prone
Rule-based (ours)	0.22	0.42	0.23
Rule-based, max	0.21	0.49	0.22
Demand ratio	0.06	0.34	0.01
Contention	0.21	0.49	0.22
Blocked-process ratio	0.29	0.03	0.31
Request rate	0.00	0.23	0.00
Free-resource threshold	0.10	0.34	0.05
Logistic regression	0.22	0.44	0.23
Boosted trees	0.36	0.58	0.43
Safety margin	0.55	0.78	0.54
Partial lookahead	0.45	0.80	0.44
Full lookahead†	0.66	0.83	0.69
Random	0.00	0.01	0.01

† common subset only.

TABLE XI
MEASURED COMPUTE PER STEP, IN MICROSECONDS AND AS A MULTIPLE OF ONE SIMULATION STEP (WHICH INCLUDES ONE SAFETY CHECK AND THE INDICATORS).

	Average over sizes		$n = 200$	
Computation	μs	× step	μs	× step
Simulation step	289	1.00	–	–
One safety check	60	0.21	–	–
Safety margin	417	1.44	479	1.66
Partial lookahead	231	0.80	264	0.91
Full lookahead	1,850	6.40	12,566	43.45

Finding 3. On the common subset the safety margin reached F1 0.72 and full lookahead 0.74, at 1.7× and 43.5× one simulation step for 200 processes in our implementation. The margin was reliably lower by 0.027 but equivalent within a post-hoc ± 0.05. Banker-based methods warned later (median 4–5 steps) than the lightweight indicators (7–13).

D. Unsafe States and Deadlock

In this simulator, all resolved unsafe episodes ended in deadlock under the bursty (29,868) and deadlock-prone (31,219) workloads, and 77% under normal (16,613 of 21,525; Table XIII). This follows partly from the design—processes eventually request their full claims, and a sequence ends at its first deadlock—and is a property of the model, not evidence about real systems (Section VI).

For the deadlock target, a comparison across the full population would mix populations, because full lookahead covers 25,862 transitions and the other methods 77,700. Table XII therefore reports every method on the common subset, with full-population F1 alongside. On identical sequences the rule-based score led with F1 0.659, followed by the free-resource threshold (0.552) and boosted trees (0.540); in the full population, blocked-process ratio and boosted trees had led (0.476). The three Banker-based methods (full lookahead 0.369, safety margin 0.342, partial lookahead 0.301) were behind every other method except request rate and random, the reverse of their standing on the unsafe target (observation A3).

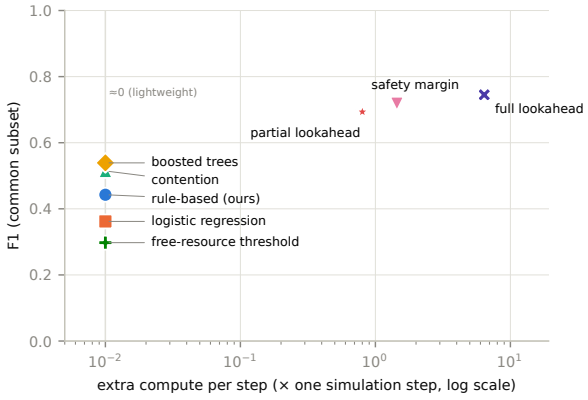


Fig. 2. F1 on the common subset against extra compute per step, averaged over system sizes (log scale).

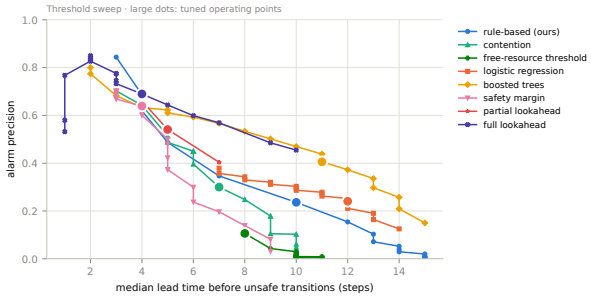


Fig. 3. Precision against median lead time along each threshold sweep.

Finding 4. Under this simulator, 77–100% of unsafe episodes ended in deadlock, partly by design. On identical sequences the rule-based score anticipated deadlocks best (F1 0.66) and the Banker-based methods were among the weakest (0.30–0.37).

E. Robustness to Misdeclared Claims

With every claim entry independently multiplied by a factor drawn from $[0.8, 1.2]$ for the indicators, while ground truth used the true claims, F1 changed by at most 0.004 for every claim-based method (Table XIV), which supports H3 as stated. The test is weak, however: the errors are independent and zero-mean, so they largely cancel in sums over processes and resource types. It says nothing about systematic misdeclaration, for example every process under-claiming by 20%, which was not tested.

Finding 5. Unbiased per-entry claim error of up to $\pm 20\%$ changed F1 by at most 0.004; biased misdeclaration was not tested.

F. Horizon Sensitivity

Scoring the same alarms against wider horizons raised every method's F1 (Table XV); the rule-based score rose from 0.108 at $H_{\max} = 5$ to 0.399 at 50: many alarms counted as false at the 20-step horizon in fact precede a transition by 21 to 50 steps. The ranking of the methods in Table XV did not change.

TABLE XII
PREDICTING DEADLOCKS UP TO 20 STEPS AHEAD. PRECISION, RECALL AND F1 ON THE COMMON SUBSET (25,862 TRANSITIONS, ALL METHODS ON THE SAME SEQUENCES); LAST COLUMN: F1 ON THE FULL POPULATION (77,700 TRANSITIONS).

	Common subset			Full
Method	Precision	Recall	F1	F1
Rule-based (ours)	0.582	0.759	0.659	0.326
Free-resource threshold	0.523	0.585	0.552	0.161
Boosted trees	0.374	0.969	0.540	0.476
Contention	0.390	0.772	0.518	0.262
Demand ratio	0.621	0.426	0.505	0.235
Blocked-process ratio	0.413	0.499	0.452	0.476
Logistic regression	0.242	0.920	0.384	0.286
Full lookahead	0.320	0.434	0.369	–
Safety margin	0.317	0.370	0.342	0.155
Partial lookahead	0.269	0.343	0.301	0.128
Request rate	0.242	0.174	0.202	0.064
Random	0.030	0.009	0.014	0.006

TABLE XIII
HOW UNSAFE EPISODES ENDED, PER WORKLOAD. AN EPISODE "RECOVERS" IF THE STATE BECOMES SAFE AGAIN.

Workload	Recovered	Deadlocked	% deadlocked
Normal	4,912	16,613	77
Bursty	0	29,868	100
Deadlock-prone	0	31,219	100

Finding 6. Widening the horizon from 5 to 50 steps raised every method's F1 (rule-based 0.11 to 0.40) without changing their ranking.

G. Scaling of the Safety Check

On random valid states, the vectorised safety check was 1.4–7.7 times faster than the textbook loop (Table XVI, Fig. 4). Its time grew with a log–log slope of 0.27 in n at $m = 20$, against 0.76 for the reference loop, far below the $\Theta(mn^2)$ worst case. This is the average-case behaviour of one implementation on these states.

Finding 7. The vectorised safety check grew far more slowly than $\Theta(mn^2)$ in these workloads (slope 0.27 at $m = 20$); this is average-case behaviour of one implementation.

H. Unexplained Observations

Six observations are not explained by anything above.

(A1) Blocked-process ratio reached F1 0.29 on normal and 0.31 on deadlock-prone workloads but 0.03 on bursty workloads, where it raised 1,402 alarms for 29,868 transitions. A plausible cause is that burst requests are capped by free units and therefore granted, so few processes block before the transition; we have not traced blocked-process counts before transitions to check this.

(A2) Boosted trees reached recall 0.985 on the deadlock target in the full population (0.992 at $H_{\max} = 50$) and 0.969 on the common subset, with precision 0.31–0.37; no other method approached this recall. Because alarms are not suppressed during unsafe episodes for this target, the model may simply alarm throughout the unsafe stretch that precedes deadlock. Restricting the analysis to alarms raised while safe would test this.

(A3) The Banker-based methods led on the unsafe target but were among the weakest methods on the deadlock target, on identical sequences (Table XII). All three are computed only on safe steps, so they are silent during the unsafe episodes that precede deadlock; this is likely a construction effect rather than a property of the signals, and computing them on unsafe steps too would resolve it.

(A4) F1 of the Banker-based methods and boosted trees fell steadily with size (full lookahead 0.756 at $n = 10$ to 0.534 at $n = 200$, safety margin 0.722 to 0.509, boosted trees 0.568

TABLE XIV
ROBUSTNESS: F1 OF THE SAME TUNED METHODS WHEN EACH DECLARED CLAIM ENTRY IS RANDOMLY OFF BY UP TO ±20% (GROUND TRUTH USES THE TRUE CLAIMS).

Method	True claims	Claims ±20%	Change
Rule-based (ours)	0.280	0.280	+0.000
Rule-based, max	0.338	0.337	−0.001
Demand ratio	0.190	0.194	+0.004
Contention	0.338	0.337	−0.001
Logistic regression	0.310	0.311	+0.002
Boosted trees	0.477	0.476	−0.001

TABLE XV
F1 WHEN AN ALARM COUNTS AS CORRECT IF AN UNSAFE TRANSITION FOLLOWS WITHIN 5, 10, 20 OR 50 STEPS (SAME ALARMS).

Method	5 steps	10	20	50
Rule-based (ours)	0.108	0.190	0.280	0.399
Contention	0.129	0.222	0.338	0.432
Boosted trees	0.178	0.330	0.477	0.597
Safety margin	0.374	0.490	0.625	0.707
Partial lookahead	0.295	0.433	0.576	0.691
Full lookahead†	0.482	0.648	0.745	0.798
Free-resource threshold	0.055	0.097	0.162	0.251
Random	0.002	0.003	0.006	0.015

† common subset only.

TABLE XVI
TIME PER BANKER'S SAFETY CHECK ON RANDOM VALID STATES: OUR VECTORISED NUMPY VERSION AGAINST THE TEXTBOOK LOOP.

n	m	Vectorised (μs)	Textbook (μs)	Speed-up
10	5	7.1	9.7	1.4
10	10	6.9	9.4	1.4
10	20	6.9	10.5	1.5
25	5	9.0	19.5	2.2
25	10	7.4	17.6	2.4
25	20	7.4	17.3	2.3
50	5	9.1	29.6	3.3
50	10	8.1	28.4	3.5
50	20	9.0	28.6	3.2
100	5	12.1	60.8	5.0
100	10	9.9	51.9	5.3
100	20	11.2	54.3	4.9
200	5	17.8	138.0	7.7
200	10	13.7	103.3	7.6
200	20	15.6	99.7	6.4

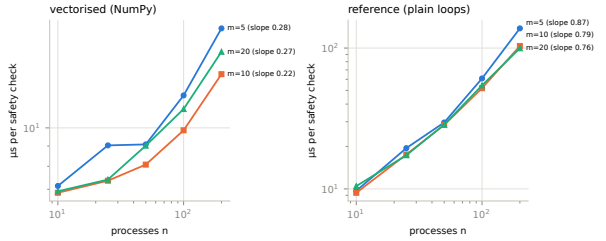


Fig. 4. Time per safety check against n (log–log).

to 0.410), and the rule-based score fell from 0.470 at $n = 25$ to 0.197 (0.433 at $n = 10$). Blocked-process ratio moved the other way, from 0.087 at $n = 10$ to 0.312 at $n = 100$ (0.270 at 200), and demand ratio and the free-resource threshold peaked at $n = 50$. Thresholds were tuned once across sizes and transition density differs by size; per-size tuning would separate the two.

(A5) Request rate scored F1 0.001 on normal and 0.000 on deadlock-prone workloads against 0.23 on bursty, and demand ratio 0.01 on deadlock-prone; three single-indicator baselines chose the top of the threshold grid (95), so their best operating point may lie outside the grid.

(A6) Logistic regression raised 5.44 false alarms per 1,000 steps on the common subset against 0.48 on the full population, above the tuning constraint of five. Thresholds tuned on all sizes may not transfer to the small systems that dominate the subset; per-size tuning (A4) would test this too.

VI. Threats to Validity

Internal. The replications reproduce test-side sampling from one generator with thresholds tuned once; they do not capture variation from re-tuning or from a different generator. The hypotheses, the equivalence margin and several analyses were chosen after the data were seen. Two pipeline bugs were fixed mid-run, the bursty workload was redesigned after calibration and the tuning split was reduced (Section IV-C).

External. The workloads are synthetic, claims are declared and the process set is fixed. One simulator and one NumPy implementation were used, so the cost results are implementation-specific (Section V-C), on one instance type in one region. The 77–100% unsafe-to-deadlock rate follows partly from the design, since processes eventually request their full claims and sequences end at the first deadlock; in systems where processes stop short of their claims, fewer unsafe states would become deadlocks.

Construct. F1 at a 20-step horizon is one operating point. "Moderate" precision is judged against clinical early warning [40]. "Unsafe" is Banker's pessimistic notion, not imminent deadlock.

Experiments not run. The claims of this paper are limited by the following: (1) real allocation traces: public cluster traces do not record per-process maximum claims or hold-and-wait sequences, so Banker ground truth cannot be derived from them; (2) biased claim misdeclaration (all claims scaled by a common factor); (3) re-tuning within each replication, which would make it a full replication; (4) two-stage gating, in which the lightweight score triggers the safety margin; (5) a second implementation of the safety check, such as an incremental test, to separate algorithmic from implementation effects; (6) a pre-registered confirmatory study with the hypotheses, δ and the number of replications fixed in advance; (7) the checks proposed for observations A1–A6.

VII. Discussion and Conclusion

Three things came out of the measurements that the simple estimate of Section I could not have told us. First, warning quality depended on the regime: lightweight indicators did well when danger built up abruptly and poorly when it built up gradually, and most methods degraded in larger systems, while blocked-process ratio improved. Second, two of our initial expectations were partly wrong: averaging indicators did worse than the strongest indicator alone, and the pilot run had suggested that full lookahead warns much later than the lightweight score (median 3 versus 7 steps), whereas in the main run the paired leads were 4 and 5. Third, the lightweight indicators hold more than a weighted sum extracts—boosted trees on the same features reached 0.48 against 0.28—and the ranking of methods depends on the population and the target: on identical sequences, the rule-based score anticipated deadlocks better than any Banker-based method.

For a practitioner, the measurements suggest a procedure: run a lightweight score continuously and, when it crosses its threshold, evaluate the safety margin, which costs a fixed seven safety checks per step and in these configurations comes within 0.03 F1 of full lookahead; for small systems, full lookahead itself is affordable. The lightweight score supplies the earlier warning and the state check the precision. This procedure follows from the components' measurements; it was not tested end to end.

For designers of warning methods, the lesson is to compare methods on identical populations, with paired replicates, and to report compute cost and lead time alongside accuracy. In this study each of these changed a conclusion: population mixing hid the deadlock-target ranking, cost separated two methods within 0.03 F1 of each other, and lead time reversed the order of the lightweight and Banker-based methods.

These results come from one simulator, synthetic workloads with declared claims, one implementation and one cloud instance. They do not establish behaviour on real allocators, under biased claims or in distributed settings.

Data and Code Availability

The simulator, configurations, analysis scripts (including `analysis/batches.py` for the batch, equivalence and common-subset analyses), per-sequence evaluation rows and a sample of per-step traces are in the authors' repository and are available from the authors on request. Configurations record seeds, configuration hashes and commit identifiers (main run `328caf6`).

References

[1] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, "Large-scale cluster management at Google with Borg," in *Proc. 10th European Conference on Computer Systems (EuroSys)*, 2015, pp. 1–17.

[2] K. R. Jayaram, V. Muthusamy, P. Dube, V. Ishakian, C. Wang, B. Herta, S. Boag, D. Arroyo, A. Tantawi, A. Verma, F. Pollok, and R. Khalaf, "FfDL: A flexible multi-tenant deep learning platform," in *Proc. 20th Int. Middleware Conference (Middleware '19)*, 2019, pp. 82–95.

[3] C.-Y. Lin, T.-A. Yeh, and J. Chou, "DRAGON: A dynamic scheduling and scaling controller for managing distributed deep learning jobs in Kubernetes cluster," in *Proc. 9th Int. Conf. on Cloud Computing and Services Science (CLOSER)*. SciTePress, 2019, pp. 569–577.

[4] Kubernetes Project, "Kubernetes v1.35: Introducing workload aware scheduling," https://kubernetes.io/blog/2025/12/29/kubernetes-v1-35-introducing-workload-aware-scheduling/, 2025.

[5] H. S. Gunawi, M. Hao, R. O. Suminto, A. Laksono, A. D. Satria, J. Adityatama, and K. J. Eliazar, "Why does the cloud stop computing? Lessons from hundreds of service outages," in *Proc. 7th ACM Symposium on Cloud Computing (SoCC)*, 2016, pp. 1–16.

[6] E. W. Dijkstra, "Cooperating sequential processes," Technological University, Eindhoven, Tech. Rep. EWD-123, 1965.

[7] ——, "The mathematics behind the Banker's algorithm (EWD 623)," https://www.cs.utexas.edu/~EWD/transcriptions/EWD06xx/EWD623.html, 1977, in *Selected Writings on Computing: A Personal Perspective*, Springer, 1982, pp. 308–312.

[8] A. N. Habermann, "Prevention of system deadlocks," *Communications of the ACM*, vol. 12, no. 7, pp. 373–377, 385, 1969.

[9] F. Belik, "Deadlock avoidance with a modified Banker's algorithm," *BIT Numerical Mathematics*, vol. 27, no. 3, pp. 290–305, 1987.

[10] R. Ewaschuk, "Monitoring distributed systems," in *Site Reliability Engineering: How Google Runs Production Systems*, B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds. O'Reilly, 2016.

[11] E. M. Gold, "Deadlock prediction: Easy and difficult cases," *SIAM Journal on Computing*, vol. 7, no. 3, pp. 320–336, 1978.

[12] T. Minoura, "Deadlock avoidance revisited," *Journal of the ACM*, vol. 29, no. 4, pp. 1023–1048, 1982.

[13] R. Finkel and H. H. Madduri, "An efficient deadlock avoidance algorithm," *Information Processing Letters*, vol. 24, no. 1, pp. 25–30, 1987.

[14] M. F. Zohora, F. Farhin, and M. S. Kaiser, "DBDAA: A real-time approach to dynamic Banker's deadlock avoidance algorithm with optimized time complexity," *PLOS ONE*, vol. 19, no. 9, p. e0310807, 2024.

[15] H. C. Tunç, U. Mathur, A. Pavlogiannis, and M. Viswanathan, "Sound dynamic deadlock prediction in linear time," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 1733–1758, 2023.

[16] C. G. Kalhauge and J. Palsberg, "Sound deadlock prediction," *Proceedings of the ACM on Programming Languages*, vol. 2, no. OOPSLA, pp. 1–29, 2018.

[17] H. Le Ngoc and H. Tran Cong, "Enhancing load balancing in cloud computing through deadlock prediction," in *Industrial Networks and Intelligent Systems (INISCOM 2023)*, ser. LNICST, vol. 531. Springer, 2023, pp. 257–274.

[18] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.

[19] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 5th ed. Pearson, 2022.

[20] E. G. Coffman, M. Elphick, and A. Shoshani, "System deadlocks," *ACM Computing Surveys*, vol. 3, no. 2, pp. 67–78, 1971.

[21] R. C. Holt, "Some deadlock properties of computer systems," *ACM Computing Surveys*, vol. 4, no. 3, pp. 179–196, 1972.

[22] ——, "Comments on prevention of system deadlocks," *Communications of the ACM*, vol. 14, no. 1, pp. 36–38, 1971.

[23] D. L. Parnas and A. N. Habermann, "Comment on deadlock preventive method," *Communications of the ACM*, vol. 15, no. 9, pp. 840–841, 1972.

[24] T. Kameda, "Testing deadlock-freedom of computer systems," *Journal of the ACM*, vol. 27, no. 2, pp. 270–280, 1980.

[25] Y. Li and R. Cook, "A new algorithm and asymptotical properties for the deadlock detection problem for computer systems with reusable resource types," in *Advances and Innovations in Systems, Computing Sciences and Software Engineering*. Springer, 2007, pp. 509–512.

[26] S.-D. Lang, "An extended Banker's algorithm for deadlock avoidance," *IEEE Transactions on Software Engineering*, vol. 25, no. 3, 1999.

[27] G. R. Andrews and G. M. Levin, "On-the-fly deadlock prevention," in *Proc. 1st ACM Symposium on Principles of Distributed Computing (PODC)*, 1982.

[28] J. W. Havender, "Avoiding deadlock in multitasking systems," *IBM Systems Journal*, vol. 7, no. 2, pp. 74–84, 1968.

[29] R. Agrawal, M. J. Carey, and D. J. DeWitt, "Deadlock detection is cheap," *ACM SIGMOD Record*, vol. 13, no. 2, pp. 19–34, 1983.

[30] K. M. Chandy, J. Misra, and L. M. Haas, "Distributed deadlock detection," *ACM Transactions on Computer Systems*, vol. 1, no. 2, pp. 144–156, 1983.

[31] E. Knapp, "Deadlock detection in distributed databases," *ACM Computing Surveys*, vol. 19, no. 4, pp. 303–328, 1987.

[32] M. Singhal, "Deadlock detection in distributed systems," *Computer*, vol. 22, no. 11, pp. 37–48, 1989.

[33] S. Bensalem and K. Havelund, "Dynamic deadlock analysis of multi-threaded programs," in *Hardware and Software Verification and Testing (HVC 2005)*, ser. LNCS. Springer, 2006, pp. 208–223.

[34] P. Joshi, C.-S. Park, K. Sen, and M. Naik, "A randomized dynamic program analysis technique for detecting real deadlocks," in *Proc. 30th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2009, pp. 110–120.

[35] J. Oluwaseyi, "AI-enhanced predictive systems for thread deadlock resolution: Early detection and prevention in cloud applications," EasyChair Preprint 14955 (not peer-reviewed), 2024. [Online]. Available: https://easychair.org/publications/preprint/fQbd/open

[36] Z. A. Banaszak and B. H. Krogh, "Deadlock avoidance in flexible manufacturing systems with concurrently competing process flows," *IEEE Transactions on Robotics and Automation*, vol. 6, no. 6, pp. 724–734, 1990.

[37] J. Ezpeleta, J. M. Colom, and J. Martínez, "A Petri net based deadlock prevention policy for flexible manufacturing systems," *IEEE Transactions on Robotics and Automation*, vol. 11, no. 2, pp. 173–184, 1995.

[38] M. P. Fanti and M. Zhou, "Deadlock control methods in automated manufacturing systems," *IEEE Transactions on Systems, Man, and Cybernetics, Part A*, vol. 34, no. 1, pp. 5–22, 2004.

[39] Z. W. Li, M. C. Zhou, and N. Q. Wu, "A survey and comparison of Petri net-based deadlock prevention policies for flexible manufacturing systems," *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, vol. 38, no. 2, pp. 173–188, 2008.

[40] S. L. Hyland, M. Faltys, M. Hüser, X. Lyu, T. Gumbsch, C. Esteban, C. Bock, M. Horn, M. Moor, B. Rieck, M. Zimmermann, D. Bodenham, K. Borgwardt, G. Rätsch, and T. M. Merz, "Early prediction of circulatory failure in the intensive care unit using machine learning," *Nature Medicine*, vol. 26, no. 3, pp. 364–373, 2020.

[41] J. Münchmeyer, D. Bindi, U. Leser, and F. Tilmann, "The transformer earthquake alerting model: A new versatile approach to earthquake early warning," *Geophysical Journal International*, vol. 225, no. 1, 2021.

[42] N. Tatbul, T. J. Lee, S. Zdonik, M. Alam, and J. Gottschlich, "Precision and recall for time series," in *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018, pp. 1924–1934.

[43] A. Lavin and S. Ahmad, "Evaluating real-time anomaly detection algorithms – the Numenta Anomaly Benchmark," in *Proc. IEEE 14th Int. Conf. on Machine Learning and Applications (ICMLA)*, 2015, pp. 38–44.

[44] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[45] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. 23rd Int. Conf. on Machine Learning (ICML)*, 2006, pp. 233–240.

[46] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

[47] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 15:1–15:58, 2009.

[48] F. Salfner, M. Lenk, and M. Malek, "A survey of online failure prediction methods," *ACM Computing Surveys*, vol. 42, no. 3, pp. 10:1–10:42, 2010.

[49] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[50] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.